面向双目视差补全的 FPGA 软硬件协同系统设计与实现

摘要

针对双目匹配在遮挡、低纹理等区域容易出现视差缺失的问题，本项目设计并实现了基于 FPGA 的视差补全软硬件协同系统。系统以团队研究形成的多尺度视差补全网络和 SESR 加速平台为技术基础，将 PC 端 SGM 视差生成、输入预处理、VU440 板端推理和结果回读评价串联为完整工作流程。针对网络同时包含卷积、转置卷积、插值与逐元素计算的特点，采用 MicroBlaze 统一调度、NPU 执行适配卷积、处理器执行其余算子的异构执行方案，并通过静态计算图、量化数据约定与张量生命周期管理实现板端部署。

实验验证包含日间场景软件推理、固定输入的软件部署参考以及 VU440 实板视差输出。对于 128×128 日间裁剪样本，FPGA 板端补全将原始 SGM 的平均绝对误差由 2.211 px 降至 0.946 px，D1 式坏点率由 15.73% 降至 3.68%。结果图显示车身原有空缺得到补全，近处车辆和背景的视差层次更加连续。测试证明了系统在固定样本上的端到端功能与板级部署可行性，给后续扩大输入规模和性能优化提供了可复现的工程基础。

关键词：双目视觉；视差补全；FPGA；MicroBlaze；软硬件协同；量化部署

第一部分 作品概述

1.1 功能与特性

本作品面向双目视觉中遮挡、低纹理等场景导致的视差缺失问题，实现了从左右目图像到完整视差图的软硬件协同处理系统。PC 端运行半全局匹配（SGM），生成初始视差及有效性掩码；VU440 FPGA 端通过 MicroBlaze 调度补全网络，由 NPU 执行适配卷积、处理器完成其余算子；结果经 DDR 回读后进行可视化与误差评价。系统支持固定样本复现、量化数据检查、板端输出对照以及实现后资源和时序分析。 [1]

1.2 应用领域

视差是双目三维感知与空间测量的重要输入。遮挡和弱纹理区域出现的无效视差会影响场景结构的连续表达。本作品通过图像引导的视差补全，面向机器人环境感知、视觉测距、三维重建及边缘视觉系统等应用需求，探索在 FPGA 平台上部署多尺度补全网络的实现方式。当前实验以固定日间图像裁剪验证系统功能，提供后续扩大输入尺寸和连续处理的工程基础。 [2-5]

1.3 主要技术特点

采用共享 RGB 特征与三级粗到细视差预测；以有效性掩码进行多尺度归一化；以静态计算图组织 145 个节点及中间数据生命周期；基于 NPU 与 MicroBlaze 混合执行卷积、插值、

转置卷积和逐元素操作；统一整数码范围、权重布局、舍入及缩放语义，配合独立整数软件参考开展对照验证。

1.4 主要性能指标

表 1 主要性能指标

指标	实测/实现后结果
固定样本	日间裁剪 128×128
板端补全整体 MAE	0.946 px（原 SGM：2.211 px）
板端 D1 式坏点率	3.68%（原 SGM：15.73%）
CPU / NPU 时钟	100 MHz
DDR 相关时钟	200 MHz
LUT / FF / BRAM / DSP 利用率	21.06% / 6.33% / 17.58% / 11.70%
时序 WNS / WHS	+0.019 ns / +0.024 ns
Vivado 片上功耗估计	7.517 W（估计置信度 Low）

1.5 主要创新点

（1）构建 SGM 初始视差与多尺度补全网络协同的 FPGA 推理系统，实现 PC 预处理、板端执行和主机评价闭环。

（2）针对已有 NPU 支持范围，采用卷积硬件加速与 MicroBlaze 软件算子协作，完成混合算子网络的整网部署。

（3）建立量化数值语义、静态图资源管理及软件/板端固定样本对照流程，支撑可复测的工程验证。

1.6 设计流程

左右目图像经 PC 端 SGM 得到带缺失的初始视差，并与 RGB 和有效性掩码组成网络输入；数据交由 VU440 板端执行多尺度补全网络；MicroBlaze 按静态节点顺序调度 NPU 与软件算子；输出写回存储并由 PC 回读，完成可视化、误差评估和硬件实现结果分析。

第二部分 系统组成及功能说明

2.1 整体介绍

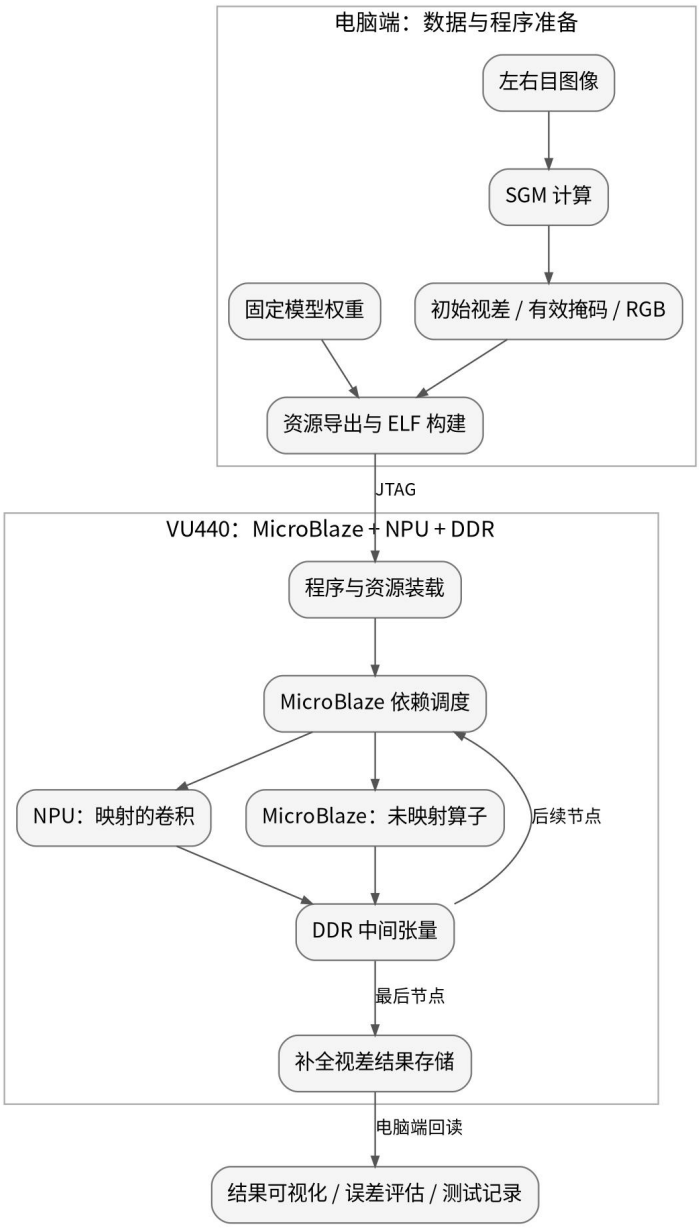


图 1 固定样本的端到端处理流程（系统级示意）。板端循环表示计算节点之间的数据依赖。

为保证板级实验输入与参考一致，部署程序采用固定输入和权重资源随可执行文件装载的方式，避免不同外部输入路径带来的额外变量。

系统采用两级处理：PC 端利用 SGM 从左右图像生成初始视差，补全网络结合 RGB、视差和有效性掩码对缺失或不可靠区域进行估计。SGM 负责生成网络输入，VU440 板端负责补全网络推理；两部分经统一输入格式衔接。 [1]

SGM 生成的带洞视差作为补全网络的输入，软件参考和板端预测采用相同裁剪区域与评价条件，使补全前后的结果能够直接比较。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

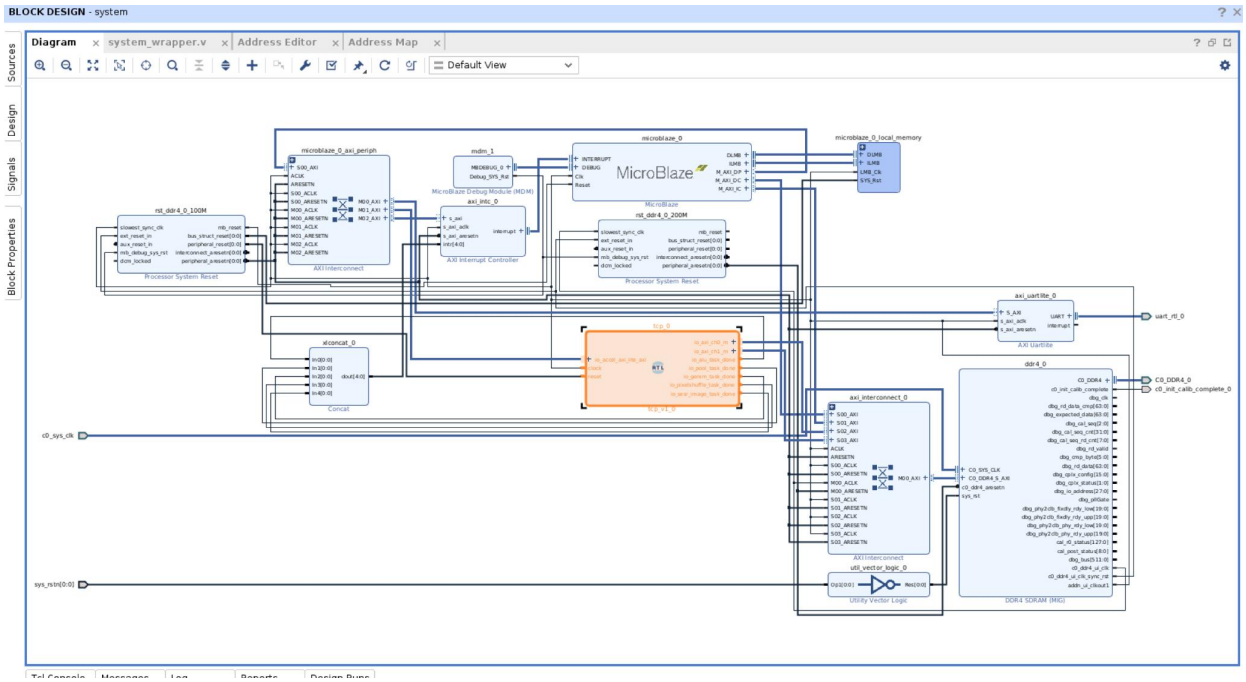


图 2 最终部署工程的 Vivado Block Design 硬件系统图。上方 MicroBlaze 经 AXI 访问外设。中央的 tcp_0 为加速模块，其两个 AXI 主接口接入右下方通向 DDR4 的互连。任务完成信号汇入中断控制器，经 MicroBlaze 的中断输入返回处理器。右侧 AXI UART Lite 用于输出运行日志。这些连接分别承担计算数据搬运、任务完成通知和软件调试。

表 2 FPGA 平台配置与约定

项目	配置或约定	技术说明
器件	xcvu440-flga2892-2-e	FPGA 器件型号，按项目硬件工程配置
处理器	MicroBlaze	负责网络调度、控制及软件算子执行
存储	DDR 地址空间 0x80000000～0xFFFFFFFF	网络输入、输出及中间数据的存储地址空间
时钟	CPU/NPU 100 MHz；DDR 相关时钟 200 MHz	CPU/NPU 设计时钟及 DDR 相关时钟，均由当前工程配置提供
串口	115200 波特率	串口通信和运行信息输出
正式工具链	Vivado / Vitis 2022.2	硬件及嵌入式软件开发工具链

指南允许采用非推荐工具版本，但要求在报告中说明并保持工程可复现。项目使用 Vivado/Vitis 2022.2，以与实际 FPGA 硬件工程和嵌入式软件环境保持一致。

针对视差补全网络的板端执行需求，本系统采用 NPU 与 MicroBlaze 协同计算、静态计算图调度和统一数值表示。下文依据最终运行版本对应的 Vivado Post-Implementation 报告，对 FPGA 资源占用、时序收敛和功耗估计进行分析。

当前工程的 CPU 与 NPU 时钟为 100 MHz，DDR 相关时钟为 200 MHz。所列资源、时序及功耗数据来自同一最终 bitstream 的实现后报告。

2.2.2 机械设计介绍

本作品采用现有 FPGA 开发板及配套散热和调试设备，不涉及独立机械机构设计。

2.2.3 电路各模块介绍

表 3 网络算子软硬件分工

运算	执行位置	划分依据
40 个普通卷积	NPU	复用已有乘加及数据传输能力
6 个转置卷积	MicroBlaze	当前采用软件实现，降低首次集成范围
插值、池化、量化、激活、加法、除法、拼接	MicroBlaze	保障完整计算图可执行
卷积输出缩放与偏置	MicroBlaze	采用部署参考明确约定的后处理顺序
图调度、资源管理及结果输出	MicroBlaze	控制依赖关系和板级接口

SESR 平台由 MicroBlaze 控制处理器和 NPU 计算单元协同组成。针对补全网络同时包含卷积、转置卷积、插值与逐元素操作的特点，采用 NPU 加速适配卷积、MicroBlaze 执行其他软件算子并协调计算顺序的分工，复用现有 FPGA 计算资源完成整网推理。 [7]

软件启动后初始化中断和运行环境，输出版本与平台信息，装入固定资源并执行网络。普通卷积通过原平台驱动提交给 NPU，其余节点调用 CPU 实现。执行完成后输出最终节点、数据长度和状态，保留结果缓冲供主机读取。

运行日志通过 BUILD 信息、节点编号、输出长度和状态码反映网络执行过程。固定部署输出为 1×128×128 的 float32 数组（65536 字节），可由主机回读用于结果评价。

软件集成了任务提交前的缓存处理、完成标志管理与有界等待，并通过阶段日志及输出检查辅助定位运行异常。固定样本的整网板端回读与效果评价见第 3.2.3 节。

当前交付源码在 completion/completion_graph_runtime.h 中，提交 NPU 前调用 Xil_DCacheFlush，计算返回后对输出执行 Xil_DCacheInvalidateRange，最终回读前调用 Xil_DCacheFlushRange 刷新结果缓冲。hw/pl_intr.cpp 的 GEMM 中断处理函数置位 gemm_task_done；hw/accel.h 的 wait_gemm_done 轮询该中断标志，最多等待 100000000 次轮询，超限打印 FAIL 并退出。此上限为轮询次数，不等同于固定毫秒超时；其他任务等待函数需分别核对。

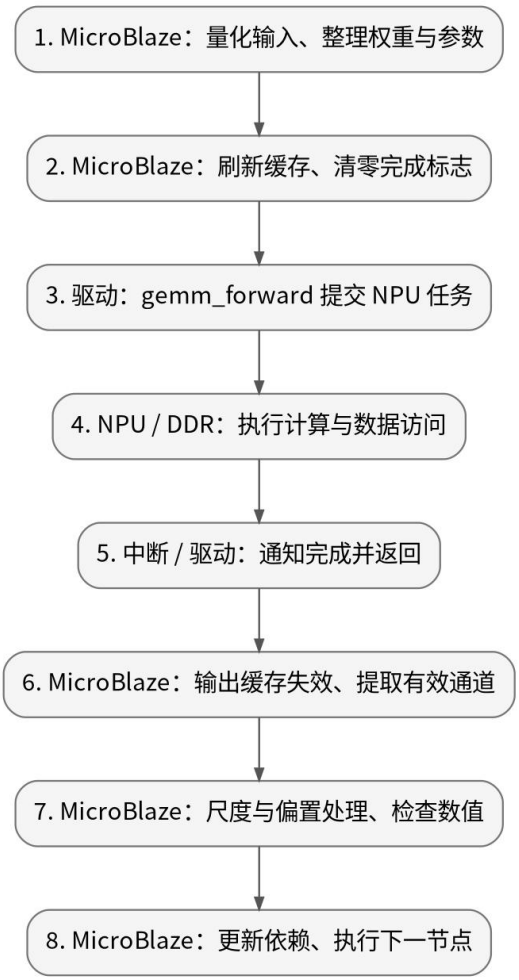


图 3 单次普通卷积的软件调用与硬件协作流程示意。

缓存处理分为两个方向：CPU 写入、NPU 读取前，需要使 CPU 修改对外部存储可见；NPU 写入、CPU 读取前，需要避免 CPU 继续使用旧缓存。最终结果回读前再次刷新结果区，保证调试器从内存读取到最新值。缓存一致性通过板端执行和 DDR 回读流程检查。

主程序还以 float 1.0 的字节序列 00 00 80 3f 检查小端 32 位浮点假设，并打印 CPU 时钟配置。每个节点执行后检查结果是否有限。NPU 输出缓冲预填 NaN，可帮助发现未写满或未正确更新的数据区域。程序结束后保持运行，防止输出缓冲因退出而失去有效性。

最终视差为 1×128×128 个 float32，共 65536 字节。每次回读采用该次运行 DONE 日志提供的输出地址。主机回读后按小端浮点解析，GT 只用于主机评价，不作为板端推理输入。

系统通过硬件周期计数接口记录网络调用区间，计时范围由调用入口和出口界定。该机制支持后续将完整网络运行时间与具体算子开销分开分析；板级效果评价使用实际回读的视差数组。

本次保存的是一次固定 128×128 样本的完成结果。串口 PERF 记录 elapsed_cycles_hi=100、lo=2370728790、assumed_npu_hz=100000000、includes_node_uart=1；计数区间覆盖 completion_graph_board_run 调用，包含节点打印和软硬件执行。

bitstream 用于配置 FPGA 硬件，XSA 为软件平台及 BSP 提供硬件描述，ELF 为 MicroBlaze 可执行程序。系统使用配套的硬件描述与可执行程序完成板级部署，保证软件调用接口与硬件配置一致。

本次上板使用 Vitis 2022.2 工程中的 system_wrapper.bit、system_wrapper.xsa 与 Release/daylight_v7_2022_app.elf，程序在 MicroBlaze 上运行并得到 STATUS 0。交付包将平台文件保存在 deployment/v7/original_platform，将本次运行 ELF 和 map 保存在 deployment/run1，并提供 SHA256 清单。当前 IDE 源码另存为 current_source_snapshot，不将后续编辑的源码等同于该 ELF 构建瞬间版本。本文实现后资源、时序及功耗截图与该次运行 bit 的精确构建对应关系仍需核对；这些截图证明其报告所对应工程的结果。

系统实现采用现有 FPGA 加速平台及对应软件运行环境。硬件配置以随项目交付的构建文件为依据，软件改动与硬件改动分别验证。

2.3 软件系统介绍

2.3.1 软件整体介绍

部署采用团队前期形成的补全网络及 epoch 36 权重。模型含 40 个普通量化卷积和 6 个量化转置卷积；另有插值、池化、激活、逐元素运算和拼接。部署静态图包含 145 个节点，当前取节点 143 作为最终结果。

表 4 固定部署样本输入输出

输入或输出	固定部署样本形状	含义
RGB	1×3×128×128	对应的左视图颜色信息
视差	1×1×128×128	SGM 输出，缺失处按输入约定处理
掩码	1×1×128×128	标识输入有效位置
输出	1×1×128×128	补全后的浮点视差，单位为像素

输入预处理保留原复现顺序。设读入视差图整数值为 R，先计算 C=1[R>0]，随后将 R≥30000 的位置清零，最后令 D=R/256。GT 也按 256 缩放到像素视差单位。RGB 先转为 float16 并除以 255，再转为推理使用的 float32；这一顺序会留下与直接 float32 归一化不同的舍入结果，应随输入资产固定。

部署保持与算法参考相同的掩码生成、异常值处理与归一化顺序，输入数组在软件与板端之间固定，确保系统比较使用一致的数据语义。

2.3.2 多尺度补全算法及数据处理

补全网络采用共享 RGB 特征提取与三级逐尺度视差预测结构。多尺度图像特征为缺失区域提供空间与边界信息，粗到细的预测过程逐步恢复视差分布；相应的池化、插值、卷积与逐元素操作则决定了 FPGA 的算子调度和中间数据组织。

表 5 多尺度网络模块与特征尺寸

模块	输入	内部特征尺寸（通道×高×宽）	阶段输出

模块	输入	内部特征尺寸（通道×高×宽）	阶段输出
RGB 编码器	3×128×128	32×128×128、32×64×64、32×32×32、32×16×16、32×8×8	五级共享颜色特征
第一视差编码器	1×32×32	32×32×32、32×16×16、32×8×8	供第一解码器融合
第一解码器	第一视差特征＋RGB 后三级	8→16→32，均为 32 通道	特征 F_1：32×32×32；预测 P_1：1×32×32
第二视差编码器	2×64×64	32×64×64、32×32×32、32×16×16	首级叠加上采样 F_1
第二解码器	第二视差特征＋对应 RGB 三级	16→32→64	特征 F_2：32×64×64；预测 P_2：1×64×64
第三视差编码器	2×128×128	32×128×128、32×64×64、32×32×32	首级叠加上采样 F_2
第三解码器	第三视差特征＋RGB 前三级	32→64→128	预测 P_3：1×128×128

每个编码级使用两个 3×3 普通卷积；下采样级的第一个卷积步长为 2。每个解码器使用两个 3×3、步长 2 的转置卷积恢复分辨率，并配合普通卷积、激活和量化。颜色与视差的同尺度特征采用逐元素相加，不是通道拼接；通道拼接发生在第二、第三阶段的视差输入处。

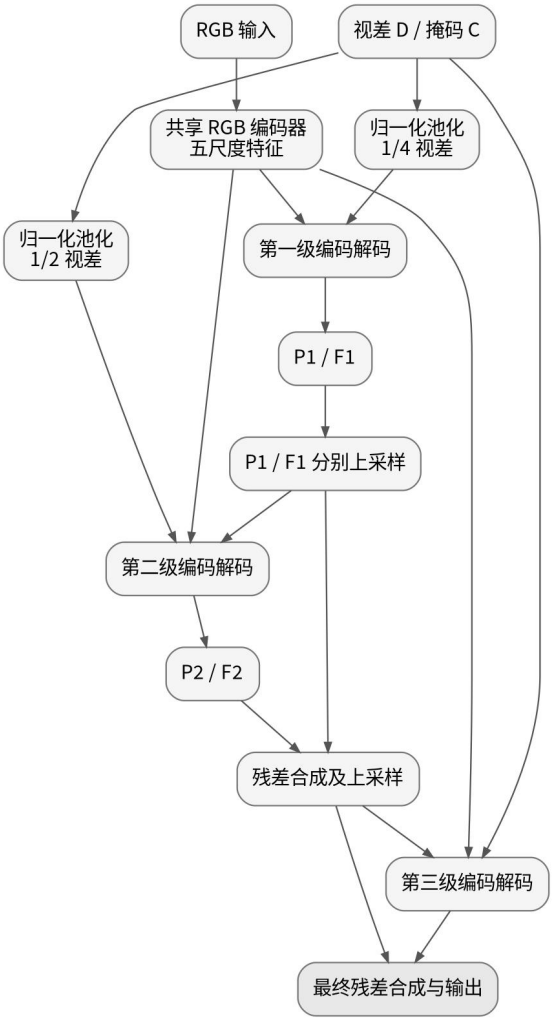


图 4 三级补全及共享 RGB 特征关系。图中上采样预测和上采样特征是不同张量；实际运算由下面公式约束。

设 D 为将无效像素置零后的输入视差，C 为对应的有效性掩码（有效为 1，无效为 0）；A_k 表示核大小与步长均为 k 的平均池化，U_2 表示 align_corners=True 的两倍双线性插值。由有效性掩码进行归一化的多尺度视差为：

$$D_{1/4} = \frac{A_4(D)}{A_4(C) + 10^{-4}}$$

$$D_{1/2} = \frac{A_2(D)}{A_2(C) + 10^{-4}}$$

其中 10^{-4} 为防止分母为零的稳定项。归一化池化可减少无效位置的零值对有效视差均值的稀释，掩码主要用于该步骤的有效性归一化。

第二阶段输入为 concat($D_{1/2}$, $U_2(P_1)$)，其第一组卷积结果叠加 $U_2(F_1)$。第三阶段先构造 $B=U_2(P_2+U_2(P_1))$，再输入 concat(D,B)，并在第一组卷积后叠加 $U_2(F_2)$。最终输出为：

$$\widehat{D} = P_3 + B$$

因此，第一阶段提供粗尺度估计，后两阶段利用已有视差、上级预测和图像特征逐级修正。报告中的“补全”包括对已知位置重新估计；当前输出没有在最后强制用原 SGM 已知像素覆盖预测。

算子数量可以从结构复核：RGB 编码器 10 个普通卷积；三个视差编码器各 6 个；三个解码器各 4 个普通卷积和 2 个转置卷积，总计普通卷积 10+18+12=40、转置卷积 6。

2.3.3 量化部署与静态执行

量化部署将网络的部分卷积映射为适合 NPU 执行的整数乘加计算，并在计算单元之间统一输入、权重、偏置、尺度、数据布局及舍入规则。NPU 负责卷积累加，MicroBlaze 侧的软件路径完成对应缩放与偏置处理；独立整数部署参考提供数值比较基准。 [6]

由于网络包含量化和逐级残差融合，部署中对浮点计算顺序、量化范围及舍入规则进行了统一规定，并以独立软件参考检查执行语义。

普通卷积输入采用逐张量尺度 s_x，权重采用逐输出通道尺度 s ,o。激活整数码 q_x 的范围为 [0,255]，权重整数码 q 为 int8。round_even 表示四舍六入、恰好位于中点时取偶数；clip 表示截断到指定范围。其量化与累加规则为：

$$q_x = \mathrm{round}_{\mathrm{even}} \left(\mathrm{clip} \left(\frac{x}{s_x}, 0,255 \right) \right)$$

$$A_{o,r,c} = \sum_{i,k_h,k_w} q_x\,(i,r',c')\,q_w(o,i,k_h,k_w)$$

$$s_o = s_x s_{w,o}$$

$$y_{o,r,c} = \mathrm{float32}\big(A_{o,r,c}\big)s_o + \hat{b}_o$$

这里的尺度与已量化反量化偏置由导出资产提供，不能在板端以未量化原偏置替代。实现保留乘尺度后再加偏置的次序，编译使用 -ffp-contract=off -fno-fast-math，避免无意改变浮点合并与重排规则。原 PyTorch 使用量化后反量化的浮点张量执行卷积，即使数学表达等价，浮点累加顺序也不保证逐值相同。

当前 COMPLETION_CPU_POSTSCALE 路径将送入 NPU 的尺度设为 1、偏置设为 0，由 NPU 返回卷积整数累加结果，再由 CPU 应用实际尺度和偏置。激活整数码在 ncnn 输入缓冲中以 float32 承载；因此“uint8 激活语义”不等于主机接口传输一定是每元素一个字节。

对于当前普通卷积配置，输入通道数最多为 32，卷积核大小为 3×3；若激活整数码绝对值不超过 255、权重整数码绝对值不超过 128，则单输出位置的整数累加绝对值满足：

$$|A| \le 32 \times 3 \times 3 \times 255 \times 128 = 9{,}400{,}320 < 2^{24}$$

因此，在上述参数范围内，整数累加值处于 float32 可精确表示的整数区间，为卷积输出的数值表示和部署参考对照提供了依据。独立软件参考使用 int64 累加；实际 RTL 累加器位宽以硬件工程为准。

图内张量使用 batch=1 的连续 CHW 布局，索引为 (cH+r)W+col。普通卷积原始权重按 OIHW 理解，转置卷积则按 IOHW，二者不可混用。调用 NPU 时，CHW 张量复制到 ncnn::Mat，通道跨度使用 cstep，回写时仅取实际 H×W 元素，跳过存储对齐区。

普通卷积输入和输出通道均按 32 对齐。设 I_a=ceil(I/32)×32、O_a=ceil(O/32)×32，则 3×3 权重存储量为 $9I_aO_a$ 字节。对于真实通道 i、o，源码使用以下对应关系读取打包权重：

$$\text{lane} = \left\lfloor \frac{i}{32} \right\rfloor \cdot 32 + 31 - (i \bmod 32)$$

$$\text{index} = \left\lfloor \frac{o}{32} \right\rfloor (9I_a \cdot 32) + \left((k_h \cdot 3 + k_w)I_a + \text{lane}\right) \cdot 32 + (o \bmod 32)$$

该映射包含输入通道组内倒序和输出通道分组。仅将 OIHW 权重顺序写入内存会造成结果错误。单通道最终输出仍按硬件布局组织权重，但只回收真实输出通道。

转置卷积采用输出位置遍历：对输出 (r,c)，只有 $r+1-k_h$ 与 $c+1-k$ 为非负偶数时，才读取输入位置 $((r+1-k_h)/2,(c+1-k)/2)$。该实现对应核 3、步长 2、padding=1、output_padding=1，输出高宽恰为输入两倍。偏置在完整累加后加入。转置卷积权重的原量化尺度沿 IOHW 第一维组织，不能照搬普通卷积的逐输出通道尺度后处理。

双线性插值严格使用 align_corners=True：输出横坐标 j 对应输入坐标 $j(W_i-1)/(W_o-1)$，纵坐标同理，再对相邻四点加权。若使用默认的另一种坐标约定，即使输出尺寸相同，边缘与内部采样也会变化。

独立激活量化节点使用 [-128,127] 的有符号范围，与卷积前的 [0,255] 不同。运行器显式实现 ties-to-even 舍入，并在反量化路径保留 ((q-v)+v)*scale 的原表达顺序，避免把原模型计算图中的浮点行为随意简化。ReLU、逐元素加法和除法保留原图顺序；拼接按 CHW 通道方向复制两个输入。

板端软件算子针对当前补全网络涉及的核大小、步长、布局及数值约定进行专门实现，以简化部署路径并保持算子行为一致。

由于本次部署网络的结构和输入尺寸固定，采用静态计算图（Static Computational Graph）预先确定计算节点、输入依赖和运算参数。板端按照图的拓扑顺序执行，不必在运行时动态构建网络；权重、输入及其他资源由统一资产索引管理。

现有资源导出、索引和哈希检查用于防止模型、输入、执行图及参考文件混用。固定输入嵌入 ELF 是当前复现方式；动态输入、完整图像分块及容量优化留待后续设计。

静态图共含 145 个节点。每个节点记录编号 id、运算类型 kind、输入引用 a/b、输出形状 c/h/w，以及整数参数 p 与尺度 scale。输入节点的编号先于使用它的节点，因而可按固定拓扑顺序执行。运算类型涵盖输入、普通卷积、转置卷积、ReLU、量化、加法、除法、拼接、池化与插值。

资源读取不访问服务器路径，而是在资产索引中匹配节点编号和字段名，同时检查预期字节数。资源缺失或长度不符会报错退出。权重资产和浮点资产采用不同的恢复方式，以保持字节与数值解释一致。

中间张量采用最后使用位置（last-use）管理：先扫描计算图中每个张量的依赖，记录它最后一次被读取的节点；执行完该节点后释放其数据缓冲，以缩短中间张量的驻留时间。最终输出则保留至整网结束，供结果回读。NPU 调用期间还需要输入输出及临时数据缓冲，内存峰值由这些实际并存的缓冲共同决定。

145 个静态节点按依赖顺序执行，其中节点 143 的张量被选作最终补全视差输出。运行器仍依据当前完整节点表执行，以保持部署计算图与导出资源的一致性。

2.3.4 验证与软件运行流程

补全质量采用平均绝对误差（MAE）、均方根误差（RMSE）以及按阈值定义的坏点率评价；部署一致性则通过输出张量的平均绝对差、最大绝对差、超容差数量和非有限值数量进行检查。所有比较均使用相同的输入裁剪、张量形状和有效真值掩码。

设 Ω 为真实视差（GT）有效像素的集合，$N=|\Omega|$ 为有效像素数，$e_i=\hat{d}_i-d_i$ 为预测视差与真实视差之差，则：

$$\mathrm{MAE}=\frac{1}{N}\sum_{i\in\Omega}|e_i|$$

$$\mathrm{RMSE}=\sqrt{\frac{1}{N}\sum_{i\in\Omega}e_i^2}$$

$$\mathrm{Bad3}=\frac{1}{N}\sum_{i\in\Omega}\mathbf{1}\left(|e_i|>3\,\mathrm{px}\right)$$

其中 $\mathbf{1}(\cdot)$ 为指示函数，Bad3 仅按绝对误差超过 3 px 计算，不同于本报告第 3.2.3 节使用的 D1 式坏点率（绝对误差同时超过 3 px 和 GT 的 5%）。原始空洞区域由输入视差为零的位置定义；空洞区域的误差只在其与 GT 有效区域的交集上统计。

可视化对比统一颜色映射及显示范围，以免色标差异造成视觉误判。GT 无效区域不参与误差统计；固定裁剪推理与整图推理因边界上下文不同而分别报告，不混用评价结果。

在相同 128×128 输入条件下，软件端分别使用 PyTorch 推理与独立整数部署参考运行网络，以检验部署运算与原网络的数值关系；该比较为板端部署提供参考。

表 6 软件参考数值一致性

样本	平均绝对差	最大绝对差	超容差数量/16384	PyTorch GT MAE	部署参考 GT MAE
000082_10	0.04165906	0.544863	15835	0.759567	0.757359
000138_10	0.00001081	0.020799	23	0.641818	0.641833
000196_10	0.00003713	0.032888	64	0.938309	0.938317

软件对比采用绝对容差 1e-5、相对容差 1e-4，同时计算相对于 GT 的 MAE。表中两种运行方式在 GT 误差上接近，说明部署计算可保持当前样本的总体补全表现。

在固定样本上，主机静态执行的最终输出与独立整数部署参考逐值一致，验证了静态计算图、资源组织和主机软件计算路径的部署一致性。

针对三组日间裁剪，采用逐节点比较定位软件推理与整数部署参考之间的数值差异。比较发现，部分差异首先出现在量化操作附近；以相同中间输入单独测试相关算子时，能够通过既定容差检查。这说明逐层比较有助于区分单个算子实现误差与上游数值扰动的传播。

例如，当量化前的数值接近半整数舍入边界时，上游极小的浮点扰动就可能使 round_even 的输出相差一个整数码；该差异又可能经后续卷积和多尺度融合传播。因此，部署验证同时关注逐值一致性与相对 GT 的预测误差：前者检验计算实现的对应关系，后者评价视差补全效果。

依据上述验证结果，部署实现统一了计算顺序、缩放系数和舍入规则，并采用独立整数软件参考作为板端计算的对照基准。

表 7 系统设计取舍

设计需求	实现方案	验证依据	工程意义
整网含现有 NPU 不直接覆盖的算子	CPU 执行转置卷积及辅助算子	网络软件路径可完整执行	完成整网算子功能覆盖
NPU 后处理与参考数值难以对齐	采用 CPU 执行缩放与偏置后处理	主机静态回归一致及已有局部验证	保持部署数值处理一致
中间张量全部保留造成存储压力	按最后使用位置释放	运行器实现可检查	减少中间张量驻留时间
输入、资源与参考容易混用	固定资产、哈希和版本检查	打包检查与主机回归	保证输入、模型资源与验证结果采用相同版本，便于结果核对和工程复现
暗场景误导对算法的整体判断	采用明确亮度规则筛选日间样本	三组软件效果及困难样本记录	提高测试样本代表性，避免由极端暗场景误判算法整体效果

本设计在现有 NPU 卷积通路上实现混合执行：非卷积操作由 MicroBlaze 完成，输入和权重采用一致的量化/布局约定，静态图按依赖次序执行，中间张量按最后使用位置回收。这些设计共同构成网络在 FPGA 平台上可执行的工程方案。

第三部分 完成情况及性能参数

3.1 整体介绍

系统已在 VU440 FPGA 平台上完成固定日间裁剪样本的整网运行，形成板端结果回读、参考比对与指标统计的完整验证流程。图 5 展示实际使用的 FPGA 系统实物及其连接情况；系统的 MicroBlaze、NPU、AXI 互连、DDR4 与中断结构则在第二部分的 Vivado Block Design 中说明。

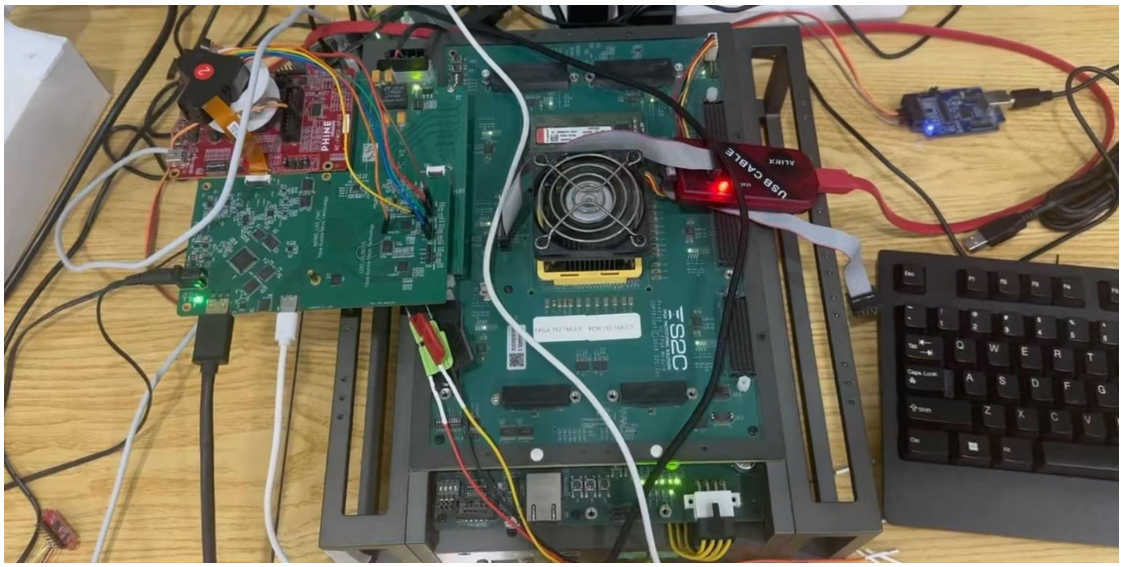


图 5 FPGA 视差补全系统实物及板级连接情况。图中展示实际使用的 FPGA 板卡、散热装置、连接线及外接调试设备。

3.2 工程成果

3.2.1 机械成果

系统采用开发板及既有连接和散热结构，无独立机械加工件；整体实物及连接见图 5。

3.2.2 电路成果

硬件成果以实际 VU440 板卡及连接照片（图 5）和 Vivado Block Design（图 2）展示；实现后资源利用率、建立/保持时间与功耗评估见第 3.3 节。系统采用 MicroBlaze 控制、NPU 卷积加速、AXI 互连与 DDR4 存储接口，构成板级推理平台。

3.2.3 软件成果

为展示算法的实际补全能力，采用三个日间样本的 1216×352 图像裁剪进行软件推理评价。结果按相同 GT 有效区域统计，MAE 的单位为像素。

表 8 日间样本软件效果

样本	输入空洞比例	输出整体 MAE	输出空洞区 MAE	输入已知区 MAE	输出已知区 MAE
000082_10	27.02%	1.1066	2.2368	1.5315	0.9387
000138_10	20.81%	0.6964	1.2016	1.4849	0.5998
000196_10	20.02%	1.2869	3.0478	1.6515	0.9559

三个日间样本均显示出输入已知视差区域的 MAE 改善，补全结果也在原空洞区域生成连续的估计。结合图像可视化，网络能够利用颜色纹理与周围有效视差恢复场景结构，为 FPGA 部署提供算法效果基础。

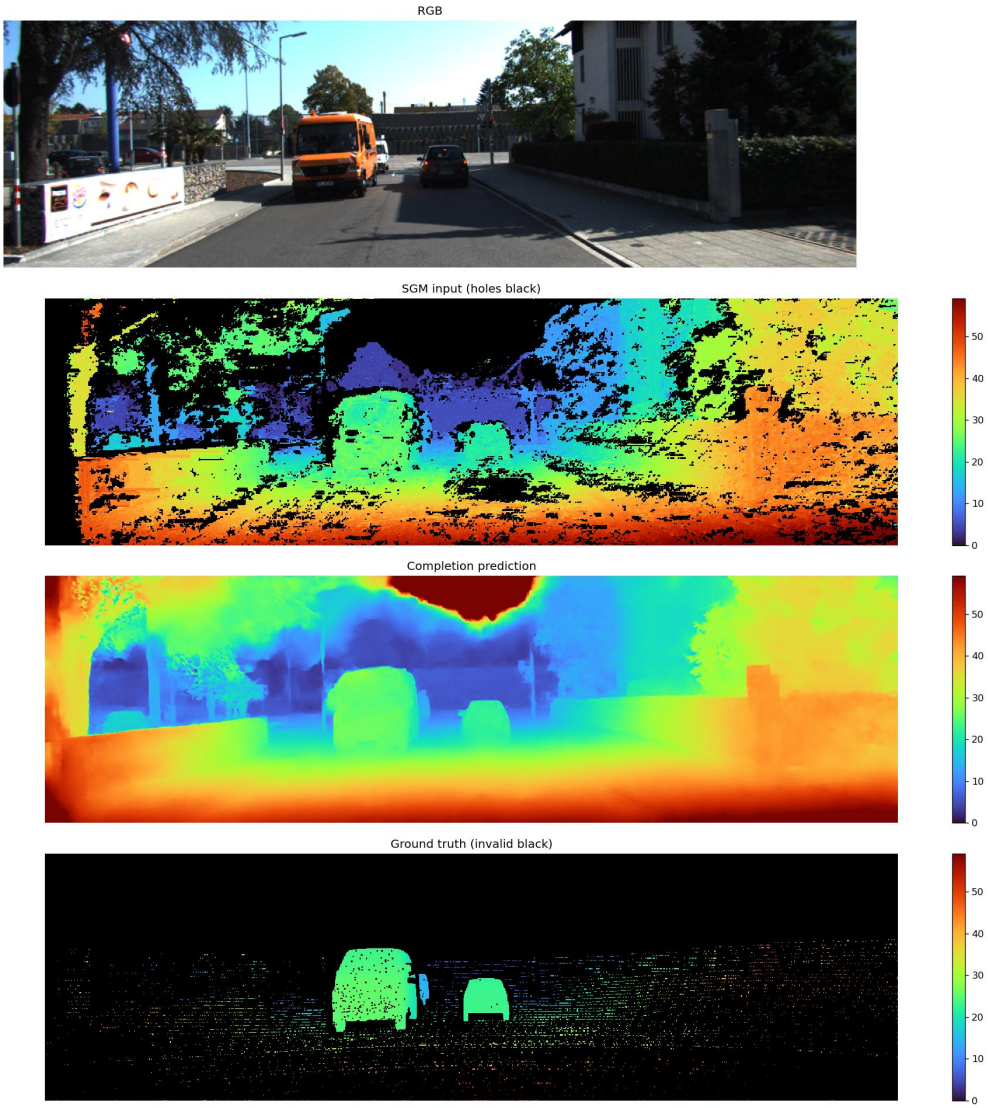


图 6 样本 000082_10 的 1216×352 软件推理可视化，自上而下为 RGB、SGM 输入、网络预测与 GT。预测结果在原始缺失区域形成连续视差，为板端部署提供直观的算法效果参考。

日间候选通过左右图像亮度、暗部比例及动态范围规则筛选，未依据算法误差挑选最佳样本。部署样本固定为 000082_10 的 x448:576、y128:256 裁剪。该裁剪的空洞比例约为 7.38%，与上表大图裁剪不同。

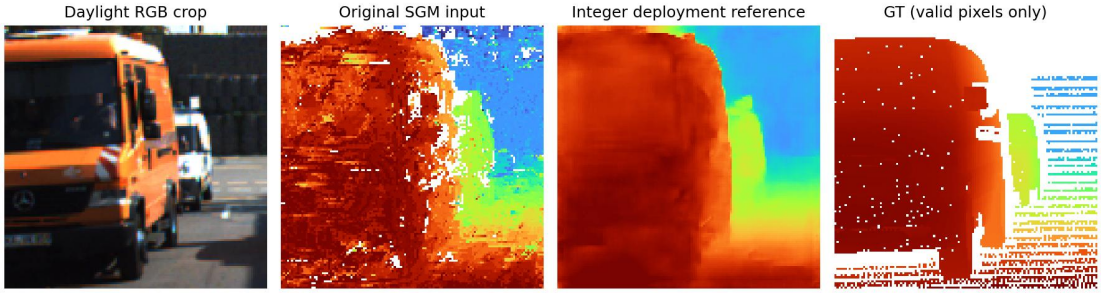


图 7 固定部署样本，从左至右为 RGB、原 SGM 输入、独立整数软件部署参考及 GT。白色表示该面板无效或无 GT 的位置。第三幅为软件参考，第 3.2.3 节展示同一裁剪的实板回读。

考虑到双目匹配对输入纹理及光照敏感，系统实验选择日间场景作为固定部署测试条件，并明确图像裁剪范围与评价掩码，以保证软件与板端结果可比。

板端视差补全输出与评价

本实验采用固定的 128×128 日间双目场景裁剪：首先由 PC 端使用 SGM 生成包含无效区域的初始视差，并与有效性掩码及对应 RGB 图像共同组成网络输入；随后将数据送入 VU440，由 MicroBlaze 调度 NPU 与软件算子执行视差补全，最后回读板端输出并与原始 SGM 进行可视化对照。图 8 展示了 RGB 图像、SGM 初始视差、FPGA 板端结果及整数软件参考。

本次原始回读文件 board_output_run1.bin 已保存，大小 65536 字节，解析为 1×128×128 的小端 float32，16384 个输出值全部有限。串口完成记录为最终节点 143、address=0x8025b360、bytes=65536、STATUS 0；地址只适用于该次运行。板端与部署参考的逐元素比较使用全部 16384 点，atol=1e-5、rtol=1e-4，MAE=0.6854828118、最大绝对差=8.900672913、超差 16333 点，严格容差比较结果为 mismatch。该比较与前述在同一 GT 有效区域评价补洞效果的口径分别保存，原始数据、比较 JSON 和 GT 指标见交付包 evidence/run1。

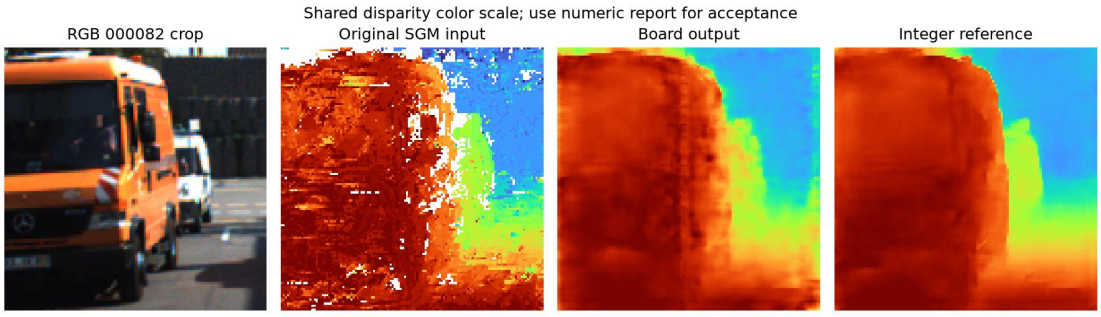


图 8 固定日间样本的视差补全可视化对比。从左至右依次为 RGB 图像、原始 SGM 视差、FPGA 板端输出与整数软件参考。

表 9 固定样本 FPGA 板端补全效果

方法	裁剪评估 MAE（px）	D1 式坏点率	原空洞且有 GT 的 451 点 MAE（px）	该 451 点坏点率
原 SGM	2.211	15.73%	—	—
软件补全（部署参考）	0.757	1.55%	0.666	1.33%
VU440 板端补全	0.946	3.68%	0.975	1.77%

从图 8 可以看到，原始 SGM 在车辆表面存在明显的视差缺失和局部不连续，影响了车身轮廓及前景、背景层次的呈现。经过板端补全后，车辆主体的视差区域更加连续，原有空洞得到填充；车辆前侧轮廓附近的局部视差变化也更容易辨认。这与网络利用 RGB 引导

信息、有效性掩码及多尺度特征融合补充缺失视差的设计目的相吻合。图像中的轮廓清晰程度属于定性观察，补全精度仍通过下述指标评价。

相较原始 SGM，FPGA 板端补全使整体 MAE 从 2.211 px 降至 0.946 px，降低约 57.2%；D1 式坏点率从 15.73% 降至 3.68%，下降 12.05 个百分点。对于原始空洞中具有有效 GT 的 451 个像素，板端输出的 MAE 为 0.975 px，坏点率为 1.77%。这表明系统在实际 FPGA 平台上实现了视差补全功能，并在所测样本上改善了原始 SGM 的视差质量。软件参考列于同一表格，作为相同输入下的部署对照。

定量结果与上述视觉观察一致：补全不仅改善了图像的连续性，也使有效 GT 区域内的预测视差整体更接近真实值。由此可见，所设计的 FPGA 软硬件协同流程能够在该固定场景中完成视差补全并改善原始 SGM 的输出质量。

3.3 特性成果

针对视差补全网络的板端执行需求，本系统采用 NPU 与 MicroBlaze 协同计算、静态计算图调度和统一数值表示。下文依据最终运行版本对应的 Vivado Post-Implementation 报告，对 FPGA 资源占用、时序收敛和功耗估计进行分析。

当前工程的 CPU 与 NPU 时钟为 100 MHz，DDR 相关时钟为 200 MHz。所列资源、时序及功耗数据来自同一最终 bitstream 的实现后报告。

表 10 最终 FPGA 实现后的资源占用（Vivado Post-Implementation）

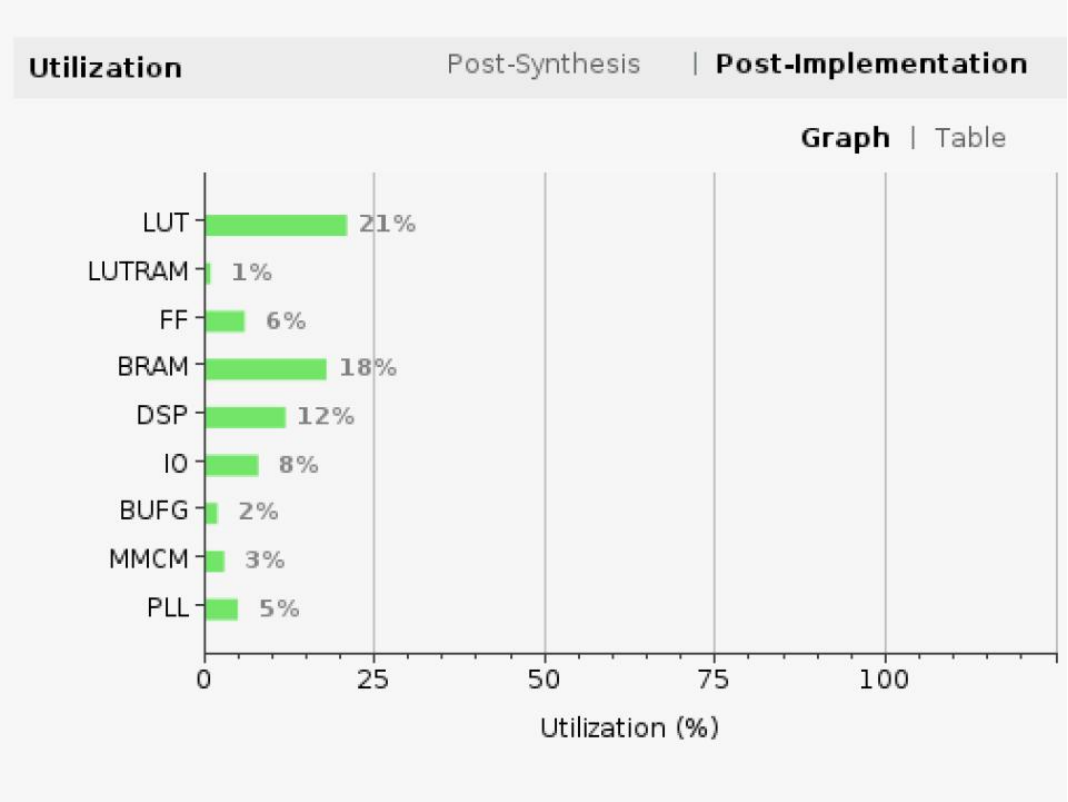
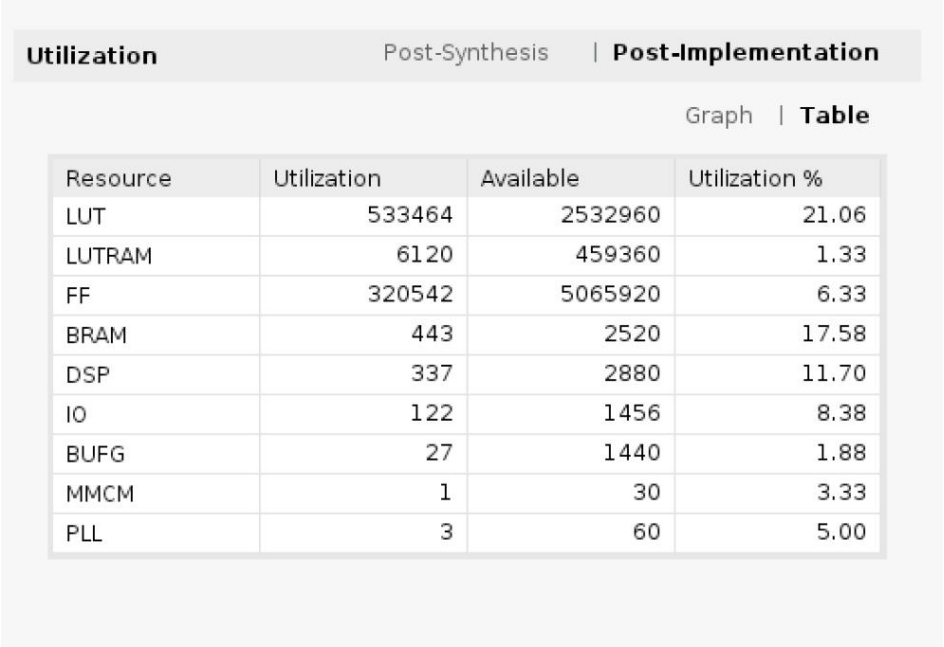


图 9 Vivado 实现后主要 FPGA 资源利用率



Utilization | Post-Synthesis | **Post-Implementation**

Graph | **Table**

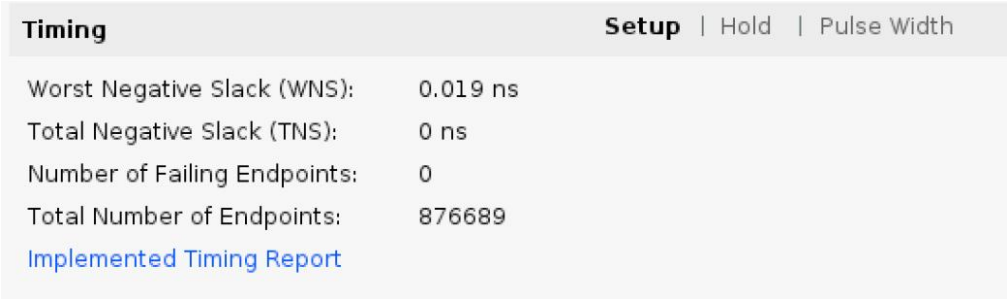
Resource	Utilization	Available	Utilization %
LUT	533464	2532960	21.06
LUTRAM	6120	459360	1.33
FF	320542	5065920	6.33
BRAM	443	2520	17.58
DSP	337	2880	11.70
IO	122	1456	8.38
BUFG	27	1440	1.88
MMCM	1	30	3.33
PLL	3	60	5.00

图 10 Vivado 实现后资源使用量及利用率统计

资源类型	已使用	可用总量	利用率
LUT	533,464	2,532,960	21.06%
LUTRAM	6,120	459,360	1.33%
FF	320,542	5,065,920	6.33%
BRAM	443	2,520	17.58%
DSP	337	2,880	11.70%
IO	122	1,456	8.38%
BUFG	27	1,440	1.88%
MMCM	1	30	3.33%
PLL	3	60	5.00%

主要资源中，LUT 利用率为 21.06%，BRAM 为 17.58%，DSP 为 11.70%，FF 为 6.33%。资源分布表明该完整实现尚有计算与片上存储容量余量，能够支撑当前视差补全网络的板端运行。

表 11 最终 FPGA 实现后的时序检查结果



Timing | **Setup** | Hold | Pulse Width

Worst Negative Slack (WNS):	0.019 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	876689

Implemented Timing Report

图 11 Vivado 实现后建立时间（Setup）时序报告

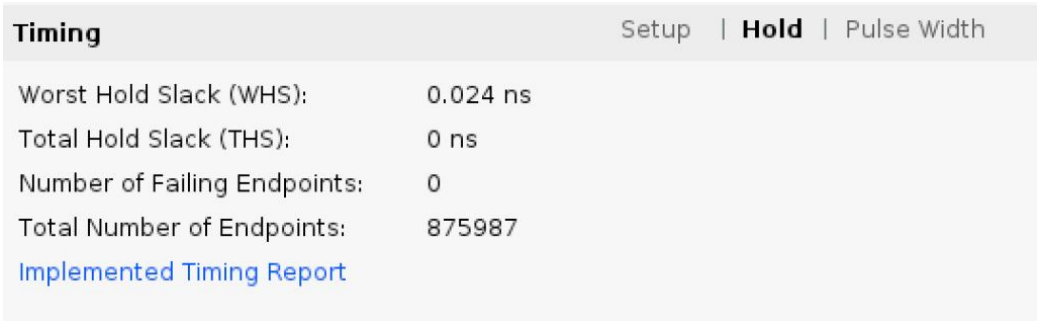


图 12 Vivado 实现后保持时间（Hold）时序报告

时序项目	结果	判断
建立时间 WNS	+0.019 ns	满足当前约束
建立时间 TNS	0 ns	无总负裕量
建立时间违例端点	0	无违例
保持时间 WHS	+0.024 ns	满足当前约束
保持时间 THS	0 ns	无总负裕量
保持时间违例端点	0	无违例

实现后最差建立时间裕量 WNS 为 +0.019 ns，最差保持时间裕量 WHS 为 +0.024 ns，且两类检查均未发现违例端点，说明设计在当前时钟约束下实现时序收敛。建立时间裕量为 0.019 ns。

表 12 最终 FPGA 实现后的功耗与热分析

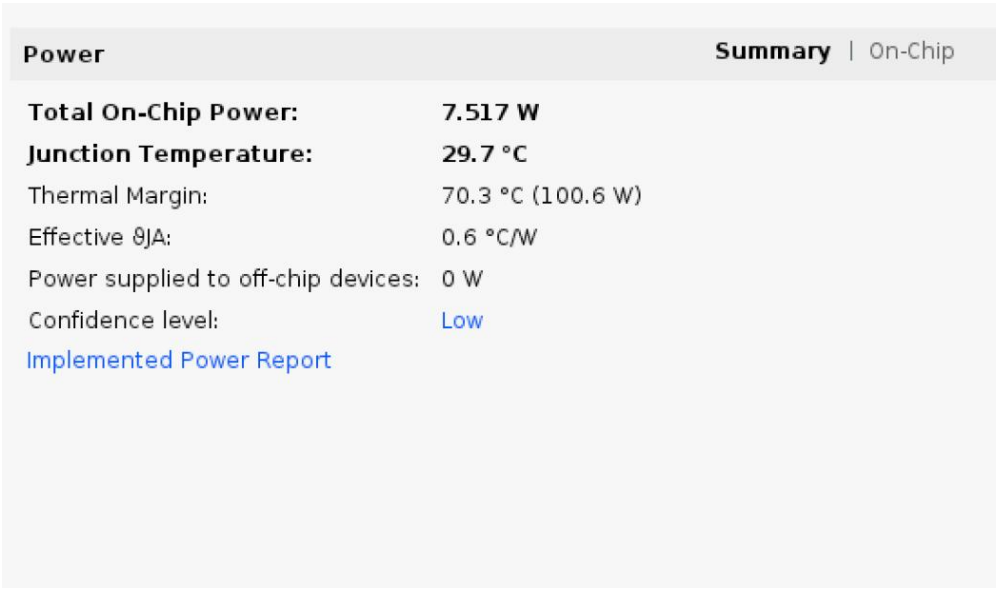


图 13 Vivado 实现后片上功耗估计报告（置信度 Low）

项目	Vivado 估计结果
片上总功耗（Total On-Chip Power）	7.517 W
结温（Junction Temperature）	29.7 °C
热裕量（Thermal Margin）	70.3 °C
估计置信度（Confidence Level）	Low

Vivado 给出的片上总功耗估计为 7.517 W，结温估计为 29.7 °C。该结果为 Vivado 片上功耗估计，报告的 Confidence level 为 Low。

综合实现结果表明，系统在所设时钟约束下完成时序收敛，主要逻辑、存储与乘加资源仍有一定容量余量，能够满足当前网络的部署需求。

第四部分 总结

4.1 可扩展之处

系统架构具备进一步扩展的基础，可在保持当前算子接口和数值协议的条件下，面向更大输入尺寸、连续图像处理及运行效率开展优化。

4.2 心得体会

本项目完成了视差补全算法与 VU440 FPGA 平台的系统集成，实现 PC 端生成初始视差、板端协同推理及主机回读评价的完整流程。多尺度网络的算子映射、量化数据约定和静态执行管理支撑了实际运行；在固定裁剪样本上，板端结果明显改善原始 SGM 的视差误差，验证了 FPGA 视差补全方案的可行性。

系统开发涉及网络算子与 FPGA 计算能力的对应关系、数据量化格式、软件和硬件之间的缓冲区管理，以及参考结果和实板结果的统一比较。通过将 SGM 预处理、静态网络执行

和结果评价拆分为清晰模块，团队建立了适用于当前网络的分层验证流程。FPGA 实板结果表明，多尺度视差补全功能已在软硬件协同环境中得到实现。

第五部分 参考文献

[1] HIRSCHMÜLLER H. Stereo processing by semiglobal matching and mutual information[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2008, 30(2): 328-341. DOI: 10.1109/TPAMI.2007.1166.

[2] GEIGER A, LENZ P, URTASUN R. Are we ready for autonomous driving? The KITTI vision benchmark suite[C]//2012 IEEE Conference on Computer Vision and Pattern Recognition. 2012: 3354-3361. DOI: 10.1109/CVPR.2012.6248074.

[3] MA F, KARAMAN S. Sparse-to-dense: Depth prediction from sparse depth samples and a single image[C]//2018 IEEE International Conference on Robotics and Automation. 2018: 1-8. DOI: 10.1109/ICRA.2018.8460184.

[4] QIU J, CUI Z, ZHANG Y, et al. DeepLiDAR: Deep surface normal guided depth prediction for outdoor scene from sparse LiDAR data and single color image[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019: 3313-3322.

[5] TANG J, TIAN F P, FENG W, et al. Learning guided convolutional network for depth completion[J]. IEEE Transactions on Image Processing, 2021, 30: 1116-1129. DOI: 10.1109/TIP.2020.3040528.

[6] JACOB B, KLIGYS S, CHEN B, et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 2704-2713. DOI: 10.1109/CVPR.2018.00286.

[7] SZE V, CHEN Y H, YANG T J, et al. Efficient processing of deep neural networks: A tutorial and survey[J]. Proceedings of the IEEE, 2017, 105(12): 2295-2329. DOI: 10.1109/JPROC.2017.2761740.

附录 A FPGA 自主选题赛道专项材料

A.1 大模型协作与实际工程决策

在视差补全系统的算法验证和 FPGA 部署过程中，团队利用大模型辅助实验诊断、数值差异分析和实现方案评估。以下按“技术问题—模型分析—实验反馈—决策结果”整理两组真实协作案例。其中技术问题经过书面化概括，模型的引号摘录保持原始回答内容，相关数据均取自项目实验记录。

A.1.1 输入样本诊断与评价方法修正

在 KITTI 000104_10 极暗隧道场景中，SGM 左右一致性检查前的视差 MAE 为 67.44 px，检查后约 96.52% 的位置无有效视差。团队曾检查匹配代价截断、聚合及掩码处理；取消截断后检查前 MAE 为 65.10 px，但空洞比例仍约 96.15%，未能解释主要异常。

模型依据新增图像与视差方向分析给出的实际回答摘录：

> *“左图灰度中位数只有 6/255，约 98.46% 的像素低于 20。”*
>
> *“按 GT 视差方向对齐，左右相关性为 0.73；反方向仅 0.16，暂不支持左右颠倒的判断。”*

团队进一步提出的技术问题（根据实际讨论整理）：异常空洞率是否受到极暗测试样本的代表性影响？长期使用单一困难样本排查 SGM，会不会导致对算法整体表现的判断发生偏差？模型据此修正了先前对测试样本的判断：

> *“这张极暗隧道图可以作为困难案例，但我没先检查原图，就长期用它作为唯一排错样本，导致前面一些实验方向不够合理。”*

团队据此将测试样本筛选前移至算法评价之前：扫描 200 对双目图像，依据亮度、暗部比例及动态范围筛出 181 对，并选取三个日间样本。筛选规则不使用模型误差或 GT 分数，避免按预测结果挑选有利样本。三个日间样本的原 SGM 空洞率分别为 26.46%、20.89%、20.40%，原空洞且有 GT 的位置经补全后的 MAE 分别为 2.237 px、1.202 px、3.048 px。此案例说明，大模型协作的价值在于结合实验反馈修正测试设计，而非直接认定 SGM 算法失效。

A.1.2 数值差异隔离与部署方案取舍

在固定日间裁剪样本上，独立整数部署参考与 PyTorch 输出的 16,384 个像素中有 15,835 个超出原设容差，但两者相对于 GT 的 MAE 分别为 0.757359 px 和 0.759567 px。团队通过逐层比较定位首个超差节点，再固定相同的上游中间张量，仅对对应部署算子进行隔离测试。模型在实验反馈后报告：

> *“首个超容差位置已做同输入验证：这几处算子在输入完全相同时都通过原容差，两个量化节点还逐值一致。”*

在同输入算子隔离验证通过后，团队将讨论集中到数值一致性与硬件执行效率之间的取舍。

> *技术问题（根据实际讨论整理）：当前超容差差异是否仍有必要修正？若继续追求与 PyTorch 逐元素一致，可能需要改变哪些运算路径，又会对 NPU 加速部署产生什么影响？*

模型据同输入隔离结果及浮点回退试验调整建议：

> *“如果为复制 PyTorch 的累加顺序，把大量计算搬回 CPU，可能既不能保证一致，又损失加速意义。刚才的浮点回退实验也没有改善结果。”*
>
> *“我的建议是：继续排查可明确修正的错误，停止以“逐值复制 PyTorch”为唯一目标。”*

最终保留 NPU 卷积计算路径，没有为追求与 PyTorch 的逐值一致而将普通卷积整体回退至 CPU。验证体系区分“部署实现与独立整数参考的一致性”和“预测输出相对 GT 的任务精度”，避免混用两类指标。该经验进一步落实为可复用的输出比较工具。

A.2 FPGA 部署结果一致性验证 Skill

团队将固定样本、数据格式约定与输出差异定位流程整理为 fpga_output_check_skill。其核心为 skill/scripts/compare.py，配套 skill/README.md、skill/SKILL.md 和 examples/vu440/

示例数据。工具不依赖特定 FPGA 型号，可在安装 Python 3.8+、NumPy 的主机上比较相同输出节点、同一样本、同一数值单位的连续张量。

使用时显式给出参考与待比较文件、张量形状、数据类型、大小端及绝对/相对容差。脚本检查文件长度、非有限值和逐元素差异，输出均值/最大绝对差、超差数量、首个超差位置及 SHA256，并将结果保存为 JSON；退出码 0、1、2 分别对应通过、数值不符和输入或参数异常。对于量化尺度、布局或字节序不同的数据，应先转换到一致表示。

可复现实例（在 skill/ 目录执行）：

```
python scripts/compare.py examples/vu440/host_daylight_final.bin
examples/vu440/deployment_postscale_final_le_f32.bin --shape 1 1 128 128 --dtype float32 --
actual-endian little --reference-endian little --atol 1e-5 --rtol 1e-4 --report host_check.json
```

随包软件实例对 1×1×128×128 个 float32 元素进行比较，检测到 16,384 个有限值、0 个超差元素，平均及最大绝对差均为 0，返回通过。该实例验证的是主机静态执行输出与独立整数部署参考的一致性。提交包另外补入真实 FPGA 原始回读张量 board_output_run1.bin 及 board_report.json；按相同容差运行比较脚本得到 mismatch，与第 3.2.3 节记录一致，软件实例与板端实例分别记录比较结果。

Skill 的提炼来源于两类实际观察：其一，异常样本会改变对算法失效范围的判断，因此必须先固定输入身份与评价口径；其二，量化边界处的小幅上游数值扰动可能在后续节点体现为整数码差异，因此应使用独立参考、统一格式并定位首个差异节点。该脚本可复用于其他 FPGA/NPU 的张量输出检查；其适用边界是同一输出语义、连续原始二进制数据及已知数值表示，不包含对任意 DMA padding、分块或带头格式的自动解析。

大模型交互记录与 Skill 使用示例随工程包归档。Skill 的使用命令、环境要求、适用范围及验证示例详见技能包 README。

A.3 工程复现与交付说明

安装 Vivado / Vitis 2022.2，准备 VU440 开发板及 JTAG、串口连接。

获取固定版本工程，核对 XSA、bitstream、输入、权重、图和参考的哈希。

交付代码以 V7 为主，README 与 docs/reproduce.md 列明平台生成、软件构建、上板运行、DDR 回读及离线验证入口；原始平台文件、已运行 ELF 与当前源码快照分开保存。包内提供文件校验、结果比较与固定样本板端证据，下列步骤说明构建、上板及回读流程。完整 Vivado 工程 49142_sesr_accel_vivado.zip 约 1.33 GB，独立放置于 GitHub Releases 下载附件，不计入比赛代码压缩包；代码与报告仓库：https://github.com/Uesugi-nuvole/SESR_submit；完整 Vivado 工程及代码压缩包下载：https://github.com/Uesugi-nuvole/SESR_submit/releases/tag/submission-2026。

由匹配 XSA 生成软件平台和 BSP，执行配置检查，构建 MicroBlaze ELF，保存日志与 map。

配置 FPGA bitstream，选择正确 MicroBlaze 调试目标，复位并装载新编译 ELF。

运行程序，保存完整串口日志，检查版本、最终节点、输出长度和状态。

按程序提供的输出地址回读 65536 字节，按小端 float32 解析 128×128 结果。

分别比较部署参考和原 PyTorch 参考，保存统计结果及展示图片。

复位并重复运行，检查输出一致性，并保存运行日志与演示材料。

第 3.2.3 节给出了按上述流程获得的固定裁剪样本板级评价结果。不同工具版本或板端配置下应保持输入与数值协议一致。

项目文件按照源码、构建与部署脚本、板级运行文件、测试输入及参考输出、技术报告与技能包进行组织，便于复现系统结构、运行步骤和图中所示的固定样本效果。